AGRI-X AI & LesionNet: IPD Presentation and Viva Guide

Oct 27, 2026 · @Raj

One-minute pitch and USP

AGRI-X AI is a farming assistant: a farmer uploads a leaf photo and gets the disease, how much of the leaf is affected, what to spray, and today's spread risk from live weather. It also recommends crops for the farmer's location and estimates yield.

Our USP is LesionNet, our own AI model. Most plant-disease models score 99% in the lab but cheat: they partly learn the background of the photo, not the disease. LesionNet is built so it cannot cheat. It first draws an outline around the diseased spots, then decides the disease only from those spots.

Analogy: a doctor who must circle the rash before naming the illness, instead of guessing from which hospital room the patient is in.

What that gives us, in three numbers:

- **Same accuracy as the best model:** 99.5% (ResNet50: 99.8%).
- **Not fooled by the background:** on a plain grey background LesionNet keeps 98.7%; ResNet50 falls to 45.1%.
- **Shows real evidence and measures severity:** its red map lands on a diseased spot 97% of the time (old models 49–64%), and its diseased-area estimate matches the true area with correlation 0.90.

Other points that set the project apart:

- No hand labelling: the training outlines are generated automatically.
- Tested honestly on a second dataset of real field photos (PlantDoc), including where it fails.
- A full working app (Streamlit) plus a research paper written in IEEE format.

Problem statement and objectives

Farmers often lose crops because a disease is spotted late or treated wrongly, and an expert is rarely nearby. Phone photos plus AI can help, but published models that score 97–99% on the PlantVillage dataset do much worse on real farms. One reason, which we measured ourselves, is that they learn shortcuts from the photo's background instead of the disease.

Problem statement: build a plant-disease diagnosis system that is accurate, looks at the actual disease, shows trustworthy evidence, and tells the farmer how severe the infection is and what to do.

Objectives

1. Train and compare four standard deep-learning models (MobileNetV2, ResNet50, EfficientNetB0, DenseNet121) on 15 tomato, potato and pepper conditions.
2. Test whether those models look at the disease or at the background.
3. Design a new model, LesionNet, that is forced to use only the diseased spots, and measure severity from them.
4. Train it without manually drawn outlines.
5. Check it on a second, real-world dataset (PlantDoc).
6. Put everything in a farmer-friendly app with treatment advice, weather risk, crop advice and yield estimates.
7. Write up the work as a research paper.

Datasets

We trained on **PlantVillage** (20,638 lab photos, 15 classes, 3 crops) and tested on **PlantDoc** (1,012 real field photos, 12 matching classes) that no model ever trained on.

Dataset 1: PlantVillage (training and main testing). Public dataset by Hughes and Salathé (2015). Photos of single leaves on plain tables, taken in controlled conditions. Images are resized to 224 × 224 pixels. Split per class: 70% train (14,440), 15% validation (3,089), 15% test (3,109), fixed random seed 42.

Crop	Condition	Photos
Tomato	Yellow leaf curl virus	3,208
Tomato	Bacterial spot	2,127
Tomato	Late blight	1,909
Tomato	Septoria leaf spot	1,771
Tomato	Spider mites	1,676
Tomato	Healthy	1,591
Pepper	Healthy	1,478
Tomato	Target spot	1,404
Potato	Early blight	1,000

Crop	Condition	Photos
Potato	Late blight	1,000
Tomato	Early blight	1,000
Pepper	Bacterial spot	997
Tomato	Leaf mould	952
Tomato	Mosaic virus	373
Potato	Healthy	152

The classes are imbalanced (largest 3,208, smallest 152, about 21×). We handled it with class weights during training.

Automatic training outlines. For LesionNet, our colour-based lesion detector made an outline for every photo (background, healthy leaf, diseased spot). 17,895 of 20,638 photos (86.7%) got a usable outline. On average lesions cover 12.8% of the leaf (median 8.8%), and the leaf covers 42.5% of the photo.

Dataset 2: PlantDoc (field test only). Public dataset by Singh et al. (2020) of real farm and internet photos with messy backgrounds, several leaves, hands, soil and blur. We used the 12 classes that match ours, 1,012 photos: pepper (bacterial spot 63, healthy 58), potato (early blight 107, late blight 104) and tomato (bacterial spot 98, early blight 77, late blight 103, leaf mould 78, Septoria 139, yellow leaf curl 74, mosaic 53, healthy 58). PlantDoc has no healthy potato, target spot or spider mite photos, so those three were not tested.

Models and parameters

We built six models: four standard ones (transfer learning from ImageNet) and two versions of our LesionNet, plus one plain baseline (A0) used only to prove where LesionNet's strength comes from.

Model	What it is	Parameters	File size	Accuracy (own test set)
MobileNetV2	Light, fast standard CNN	2.63 M	27.6 MB	91.6%
ResNet50	Deep CNN with skip connections	24.78 M	259.2 MB	99.5%
EfficientNetB0	Balanced, efficient CNN	4.39 M	31.5 MB	95.8%

Model	What it is	Parameters	File size	Accuracy (own test set)
DenseNet121	CNN that reuses all earlier features	7.34 M	38.1 MB	96.0%
LesionNet (ours)	EfficientNetB0 + lesion outline head + lesion-gated pooling	4.95 M	94.2 MB	99.55%
LesionNet-Mobile (ours)	Same idea on MobileNetV2	3.35 M	63.7 MB	95.98%
A0 plain EfficientNet	LesionNet's backbone trained the ordinary way (ablation only)	4.88 M	—	99.6%

The four standard models were evaluated on their own original test split. LesionNet's numbers are on our 3,109-photo test split. On that same split the standard models scored 93.0%, 99.8%, 96.4% and 96.2%.

Training settings: standard models

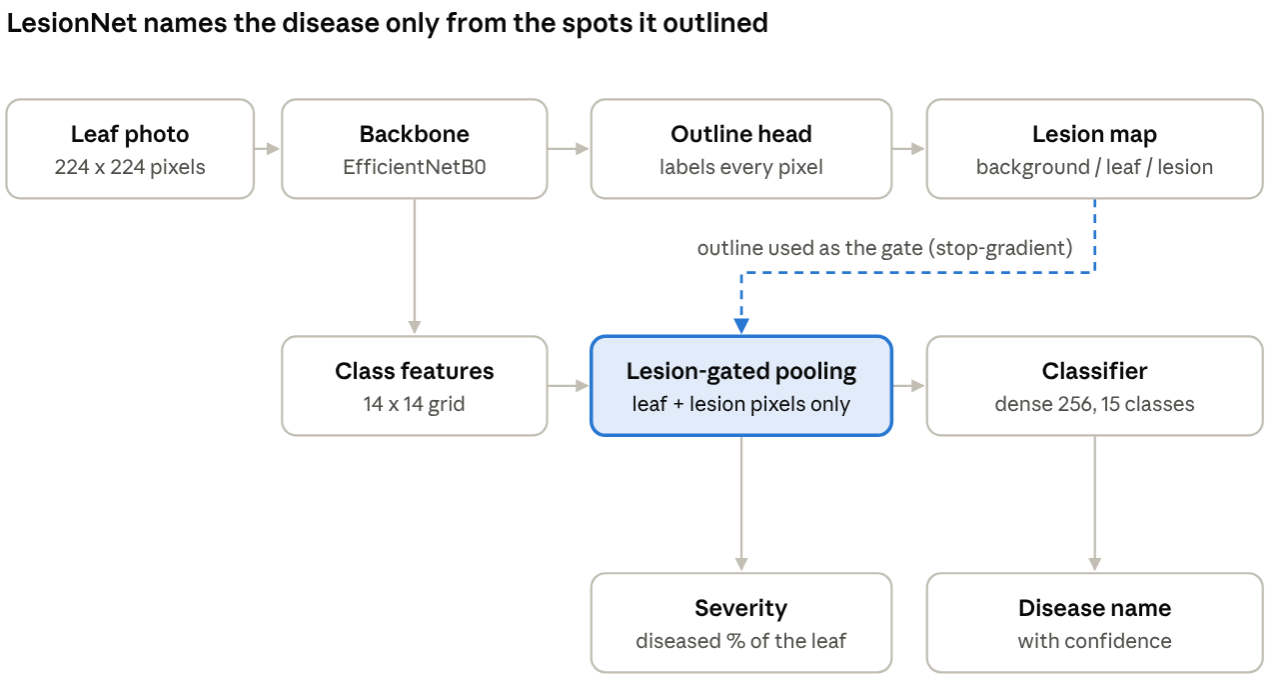
- Input 224 × 224, batch size 32, 70/15/15 split, seed 42.
- Phase 1: 5 epochs with the pretrained backbone frozen, learning rate 0.0001.
- Phase 2: 15 epochs fine-tuning the last 50 layers, learning rate 0.00001.
- Head: global average pooling → batch normalisation → dense layers → dropout 0.3 → softmax (15 classes).

Training settings: LesionNet

- Optimizer Adam. Phase 1: 3 epochs, backbone frozen, learning rate 0.001. Phase 2: 15 epochs, whole network, cosine decay from 0.0001 to 0.000001.
- Batch size 32, mixed precision (16-bit) on the GPU, early stopping with patience 5, best epoch kept by validation accuracy.
- Loss = classification loss + 1 × outline (segmentation) loss. Diseased pixels count 3× in the outline loss because they are rare. Class weights between 0.5 and 3 fix class imbalance.
- Augmentation: flips, 90° rotations, brightness ±0.12, contrast and saturation 0.85–1.15, hue ±0.02, and **background swap** (50% of photos get a random new background).
- Outline map size 56 × 56; class features 14 × 14 × 256; dropout 0.3; dense layer 256 units.

Hardware and time: NVIDIA RTX 4050 Laptop GPU (6 GB), Ubuntu on WSL2, TensorFlow/Keras 3. LesionNet trained in 76 minutes, the A0 baseline in 42 minutes. LesionNet-Mobile stopped at epoch 10 of 18 because of a GPU driver crash; we use its best saved epoch (96.2% validation).

How LesionNet works



LesionNet architecture · from research/lesion_net/lesionnet.py

One photo goes in; two branches come out of the shared backbone, and the outline from the top branch decides which pixels the bottom branch may use.

1. **Backbone (EfficientNetB0, pretrained on ImageNet):** turns the photo into feature maps at three sizes. Think of it as the eyes.
2. **Outline head:** a small FPN-style decoder that labels every pixel (on a 56 × 56 grid) as background, healthy leaf or lesion. This is the lesion map the farmer sees.
3. **Lesion-gated pooling (the new idea):** class features are averaged only over pixels the outline marked as lesion, and separately over leaf pixels, plus the lesion area ratio. Background pixels never reach the decision. Analogy: a judge who may only look at evidence that was formally submitted, not at the courtroom decor.
4. **Stop-gradient on the gate:** the classifier cannot change the outline, so it cannot "cheat" by stretching the outline over the background.
5. **Classifier:** dropout 0.3, a 256-unit layer and softmax over 15 classes give the disease and its confidence.

6. **Severity:** diseased pixels ÷ leaf pixels from the same map, graded mild / moderate / severe.

How it was trained without hand labels: our colour-based detector compared each pixel with the healthy green of the same leaf (brown or dark = dead tissue, yellow = yellowing) and used GrabCut to separate leaf from background. Healthy-class photos were marked fully healthy. Unsure photos were used only for classification. Half the training photos also got a random fake background (background swap) so the model could not learn "this table colour = this disease".

Results

LesionNet ties the best model on normal photos (99.5% vs 99.8%) but stays at 98.7% when the background is changed, while the others fall to 45–81%. All results are on the 3,109 held-out test photos unless stated.

1. Background tests (does the model use the leaf or the table?) We kept each test leaf and changed only its surroundings:

- **Grey background:** leaf pasted onto plain grey.
- **Swapped background:** leaf pasted onto a photo from another class.
- **Background only:** leaf removed completely; pure guessing would score 6.7% (1 in 15).

Model	Normal	Grey bg	Swapped bg	Background only (lower is better)
MobileNetV2	93.0%	76.0%	71.6%	19.1%
ResNet50	99.8%	45.1%	60.0%	37.4%
EfficientNetB0	96.4%	81.2%	62.3%	17.8%
DenseNet121	96.2%	56.4%	57.6%	28.9%
A0 plain EfficientNet	99.6%	67.5%	63.7%	32.7%
LesionNet-Mobile	96.0%	94.2%	73.2%	20.3%
LesionNet	99.5%	**98.7%**	**81.8%**	19.5%

ResNet50 guesses the right disease from the background alone 37.4% of the time, 5.6× random. That is the "cheating" we found.

2. Does the evidence sit on the disease? (2,222 diseased test photos; old models explained with Grad-CAM++, LesionNet with its own map)

Measure	Old models	LesionNet
Share of evidence on diseased spots	7–12%	44%
Share of evidence on background	25–49%	6.7%
Strongest point lands on a lesion (pointing game)	49–64%	97%
Overlap with lesion outline (IoU)	0.08–0.13	0.50
Focus on lesions vs healthy leaf (1 = no preference)	0.96–1.22	6.06

3. Severity. LesionNet's estimated diseased area matches the measured area with Spearman ρ = 0.90, average error 5.7 percentage points, and it reports only 0.55% disease on healthy leaves. Heatmap-based area from the old models had no link to real severity ($|\rho| \leqslant 0.13$) and showed 66–100% "affected" area even on healthy leaves.

4. Ablation (proof the design matters). A0 uses exactly LesionNet's backbone, data and training but no outline and no gating. It is just as accurate on normal photos (99.6%) but drops to 67.5% on grey backgrounds. So LesionNet makes 25× fewer errors there because of its design, not its backbone.

5. Real field photos (PlantDoc, 12 classes, guessing = 8.3%). Every model drops a lot: MobileNetV2 23.6%, A0 23.1%, LesionNet-Mobile 21.6%, LesionNet 20.8%, EfficientNetB0 20.2%, DenseNet121 20.1%, ResNet50 19.6%. The differences are within the margin of error (about ±2.6 points). The models usually get the crop right (62–71%) but not the disease. This is an honest limitation: lab photos alone are not enough for the field.

6. Full metrics. LesionNet: accuracy 99.55%, precision 99.55%, recall 99.55%, F1 99.55%, AUC 1.00. LesionNet-Mobile: accuracy 95.98%, precision 96.12%, recall 95.98%, F1 95.96%, AUC 0.9995.

7. Speed and size. LesionNet adds only 0.56 M parameters (+13%) to EfficientNetB0 and runs in about the same time (50.7 vs 50.3 ms per photo on a CPU, measured on the same machine). LesionNet-Mobile runs in 33 ms. LesionNet has a fifth of ResNet50's parameters at about half its time.

The AGRI-X app

AGRI-X is a Streamlit web app with seven pages; LesionNet is the default model on the Disease Detection page.

Page	What the user gets
Home	Overview, farm location and today's weather
Disease Detection	Upload a leaf photo, pick a model (LesionNet recommended). Shows disease, confidence, severity (mild under 10% of leaf, moderate 10–25%, severe over 25%), red map of diseased spots, per-spot list, top-5 other possibilities, treatment advice and weather spread risk
Crop Advisor	Location by GPS, city search or coordinates; weather and 7-day forecast; soil (SoilGrids or regional data); 40 crops ranked by season, temperature, water, soil, pH and region; MSP-based profit; yield forecast with a what-if simulator
Farmer Assistant	Chatbot that answers using the latest diagnosis, weather and location
Research Dashboard	LesionNet study, training curves, confusion matrices, ROC curves, explanation maps, metrics
Model Comparison	All six models side by side, plus the background-robustness charts
Reports & About	Download a field report (CSV or HTML), the research paper, dataset explorer, method and references

Crop Advisor scoring (out of 100): season 20, temperature 25, water 20, soil 15, pH 10, region 10. Seasons: Kharif (May–Aug), Rabi (Sep–Jan), Zaid (Feb–Apr). MSPs come from the official Kharif 2026-27 and Rabi 2027-28 lists (e.g. paddy ₹2,441, wheat ₹2,610 per quintal).

Data sources used live: Open-Meteo (weather and climate history), ISRIC SoilGrids (soil), OpenStreetMap Nominatim (place search). When a service is offline the app falls back to built-in regional data for all Indian states.

Tech stack and architecture

Everything is Python: TensorFlow/Keras 3 for the models, Streamlit for the app, OpenCV for image processing.

Layer	Tools
Deep learning	TensorFlow, Keras 3, mixed precision on an RTX 4050 GPU (WSL2 Ubuntu)
Image processing	OpenCV (colour spaces, GrabCut, morphology), Pillow, NumPy
Evaluation	scikit-learn (metrics, ROC, confusion matrix), pandas
Charts	Plotly (app), Matplotlib (reports)
App	Streamlit, custom CSS theme, streamlit-js-eval for GPS
External data	Open-Meteo, ISRIC SoilGrids, Nominatim
Paper	LaTeX (IEEE conference template)
Version control	Git and GitHub, large model files through Git LFS

Main code files: `app.py` (the app), `lesionnet_inference.py` (runs LesionNet in the app), `research/lesion_net/lesionnet.py` (the model), `train_lesionnet.py`, `make_pseudo_masks.py`, `evaluate.py`, `eval_external.py`, `lesion_segmentation.py` (colour-based lesion detector), `crop_advisor.py`, `weather_service.py`, `risk_engine.py`, `treatment_engine.py`, `farmer_chatbot.py`.

How a diagnosis flows: photo → resized to 224 × 224 → LesionNet → disease + lesion map → diseased % → severity grade → treatment advice + weather risk → field report.

Limitations and future work

Be upfront about these in the viva; examiners respect a team that knows its weak points.

Limitations

1. **Field photos:** every model, LesionNet included, scores only about 20–24% on real farm photos. Lab photos teach a single leaf on a plain table; farms have many leaves, soil, sunlight and blur.
2. **Automatic outlines are colour-based:** they miss faint symptoms (viruses, mites), so the "evidence on disease" scores partly use outlines LesionNet learned from. The background tests do not depend on them and are the fair comparison.
3. **One training run each:** results are from a single random seed; LesionNet-Mobile stopped early at epoch 10 of 18 due to a GPU crash.

4. **Old models' split:** the four standard models were trained earlier on a different split, so they may have seen some of our test photos. That can only make them look better; A0 is the clean comparison.
5. **Some background signal remains:** LesionNet still gets 19.5% right from background only (guessing = 6.7%), likely because features near the leaf edge "see" a bit of background.
6. **Only 3 crops and 15 conditions.**

Future work

1. Fine-tune on part of PlantDoc field photos so the app works outside the lab.
2. Handle photos with several leaves.
3. Add more crops common in India (rice, wheat, cotton, chilli).
4. Run the remaining ablations and 3 seeds per model for the paper.
5. Test against hand-drawn lesion outlines (e.g. PlantSeg dataset).
6. Ship LesionNet-Mobile in a phone app that works offline, with regional languages.

Likely viva questions

Short answers you can say in your own words.

Q1. What is new in your project? Many apps already detect plant disease. Most of them are accurate in the lab but partly learn the photo background. We proved that (ResNet50 drops from 99.8% to 45.1% on a grey background) and built LesionNet, which can only use the diseased spots. It keeps 98.7%.

Q2. Why did you choose PlantVillage? It is the standard public benchmark (20,638 images for our 15 classes), so our results can be compared with other papers. Its weakness, lab backgrounds, is exactly what we tested.

Q3. What is transfer learning? Starting from a model already trained on ImageNet (1.2 million everyday photos) and retraining it on leaves. It learns faster and needs less data, like a doctor who already knows anatomy learning a new specialty.

Q4. What is lesion-gated pooling? Pooling means summarising a feature map into one vector. Normal models average over the whole image. We average only over pixels our model marked as leaf or lesion, so background pixels never reach the decision.

Q5. Why "stop-gradient" on the gate? So the classifier cannot secretly widen the outline onto the background to cheat. Only the outline training decides where the outline goes.

Q6. How did you get outlines without drawing them by hand? Our colour-based detector compares each pixel with the healthy green of that same leaf: brown or dark = dead tissue, yellow = yellowing. It also uses GrabCut to separate the leaf from the background.

Healthy-class photos are marked as fully healthy. Photos where the detector was unsure were used only for classification.

Q7. What is Grad-CAM? A heatmap showing roughly where a normal model looked. It is computed after the decision and is coarse (7 × 7 grid). LesionNet's map is the actual input to its decision, so it is more trustworthy.

Q8. How do you measure severity? Diseased pixels ÷ (leaf + diseased pixels) from LesionNet's map. Under 10% mild, 10–25% moderate, over 25% severe. It matches the measured area with correlation 0.90.

Q9. Why is accuracy on PlantDoc so low? PlantDoc is real farm photos with many leaves, soil and bad lighting, which none of the models saw in training. All models drop to about 20–24%. Fixing it needs training on field photos; that is our next step.

Q10. How do you know the improvement comes from your design and not the backbone? Ablation A0: same EfficientNetB0 backbone, same data and training, but normal pooling. It gets 99.6% normally but only 67.5% on grey backgrounds versus LesionNet's 98.7%.

Q11. What are precision, recall, F1 and AUC? Precision: of the leaves we called diseased X, how many really were. Recall: of all real X leaves, how many we caught. F1: their balance. AUC: how well the model ranks right answers above wrong ones (1.0 = perfect). LesionNet: 99.55% / 99.55% / 99.55% / 1.00.

Q12. Why is the dataset imbalanced and how did you handle it? Yellow leaf curl has 3,208 photos, healthy potato only 152. We gave rare classes higher weight in the loss (between 0.5× and 3×) and used a stratified split so every class appears in train, validation and test.

Q13. Why 224 × 224 images? It is the input size the ImageNet-pretrained backbones were built for.

Q14. What is mixed precision? Doing most maths in 16-bit numbers on the GPU instead of 32-bit. It roughly halves memory and speeds up training on our 6 GB GPU.

Q15. How does the app decide treatment and risk? Treatment comes from a knowledge base per disease (chemical, organic, irrigation, fertiliser). Spread risk combines live temperature, humidity and rainfall from Open-Meteo with each disease's favourable conditions.

Q16. Which model would you deploy on a phone? LesionNet-Mobile: 3.35 M parameters, about 33 ms per photo on a CPU, and still 94.2% on a new background.

Q17. What problems did you face? The GPU was disabled in the laptop's power mode (fixed by switching to Hybrid); TensorFlow's XLA compiler crashed with mixed precision (we turned XLA off); a GPU driver crash stopped one training run; and we found a Grad-

CAM bug that silently gave wrong heatmaps for ResNet50 and DenseNet121, which we fixed and documented.

Q18. Did you use AI tools? Answer honestly and say how they were used and that the team checked all results. Agree the exact wording with your guide beforehand.

Cheat sheet and glossary

Numbers to remember

What	Number
Training dataset	PlantVillage, 20,638 photos, 15 classes, 3 crops
Split	14,440 train / 3,089 validation / 3,109 test (70/15/15)
Field test dataset	PlantDoc, 1,012 photos, 12 classes
LesionNet accuracy	99.55% (F1 99.55%, AUC 1.00)
LesionNet on grey background	98.7% (ResNet50 45.1%, plain EfficientNet 67.5%)
LesionNet on swapped background	81.8% (old models 57.6–71.6%)
Evidence on diseased spots	44% (old models 7–12%)
Points at a lesion	97% (old models 49–64%)
Severity correlation	0.90 (old heatmaps ≤ 0.13)
Field photo accuracy	about 21% for LesionNet (all models 19.6–23.6%)
LesionNet size	4.95 M parameters, about 50 ms per photo on CPU
Training time	76 minutes on an RTX 4050 laptop GPU
Automatic outlines	17,895 of 20,638 photos (86.7%)

Glossary

- **CNN (convolutional neural network):** a model that learns visual patterns by sliding small filters over an image.
- **Backbone:** the pretrained feature extractor inside a model (here EfficientNetB0 or MobileNetV2).
- **Segmentation:** labelling every pixel (here: background, healthy leaf, lesion).
- **Pseudo-mask:** an automatically made outline used as a training label instead of a hand-drawn one.

- **Shortcut learning:** a model getting the right answer for the wrong reason, e.g. from the background.
- **Ablation:** removing one part of a model to show what that part contributes.
- **Grad-CAM / Grad-CAM++:** heatmaps of where a normal CNN looked.
- **Pointing game:** does the hottest point of the map land on a real lesion?
- **IoU (intersection over union):** overlap between two areas, 0 to 1.
- **Spearman ρ:** how well two rankings agree, −1 to 1.
- **Epoch:** one full pass over the training data.
- **Fine-tuning:** retraining the pretrained layers slowly on the new task.
- **Data augmentation:** random flips, rotations, colour changes and background swaps that make the model more robust.

Team: Raj Naik, Dhruv Patel, Shreya Mehta, Parshva Mathukia. **Guide:** Prof. Prachi Tawde, Department of Information Technology, Dwarkadas J. Sanghvi College of Engineering, Mumbai.